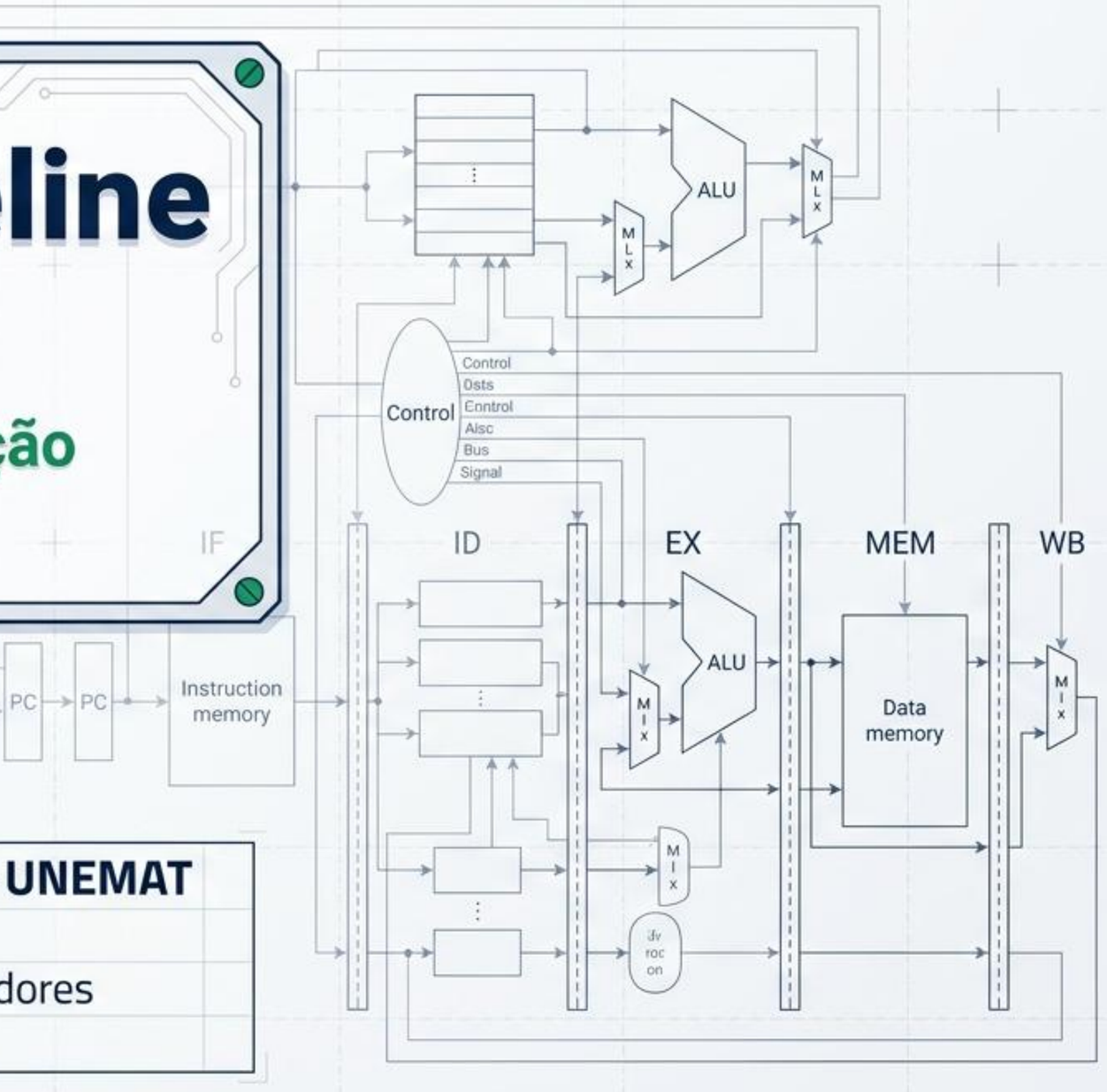
Unidade — Pipeline e Desempenho

Paralelismo no Nível de Instrução e Avaliação Quantitativa



Universidade do Estado de Mato Grosso — UNEMAT

Curso: Bacharelado em Ciência da Computação

Disciplina: Arquitetura e Organização de Computadores

Prof. Me. Juvenal Silva Neto

Gemini Notebook

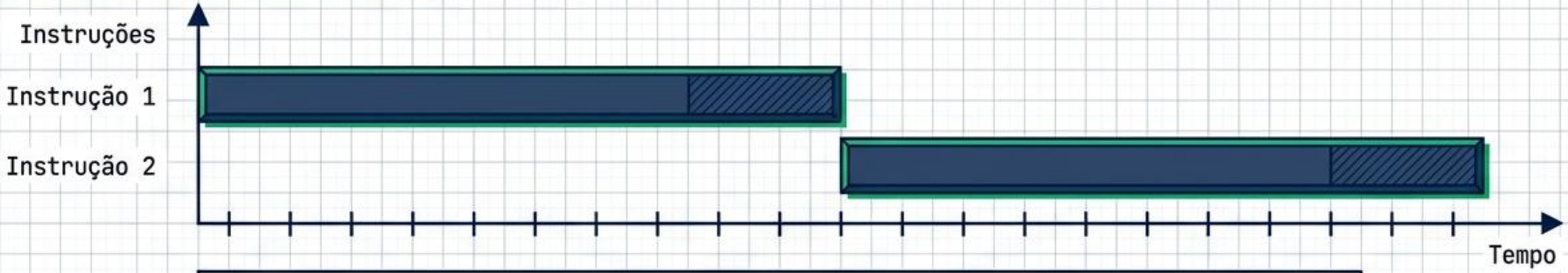
Desperdício de hardware:

enquanto a ULA calcula, as outras unidades ficam ociosas.



Como uma CPU executa várias instruções simultaneamente sem misturar os dados?

O Paradoxo Sequencial

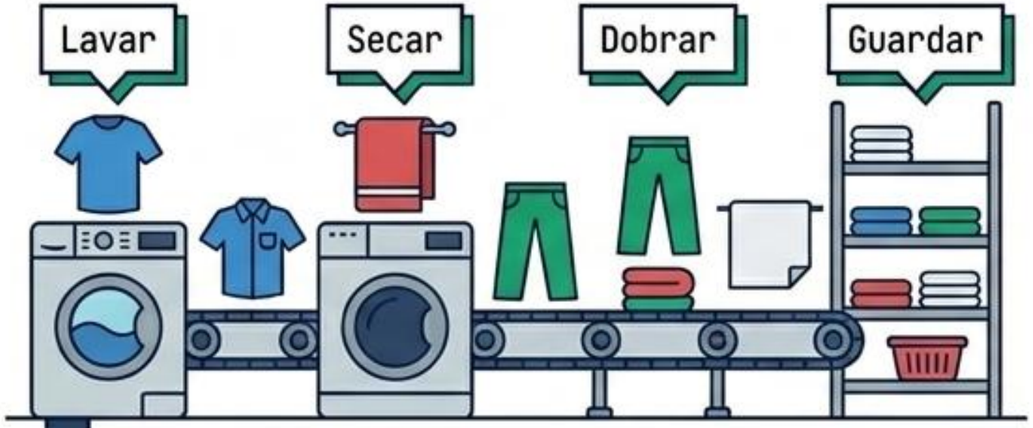


O ciclo clássico processa uma instrução inteira de cada vez. O limite de desempenho é ditado pela instrução mais lenta. Para escalar, precisamos de sobreposição.

Gemini Notebook

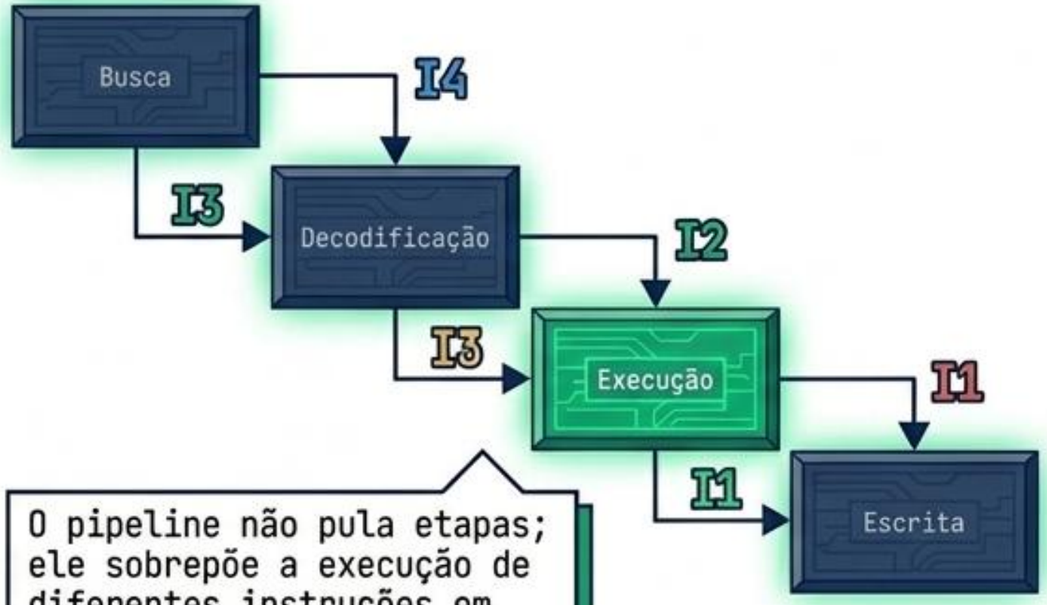
A Inovação: A Linha de Montagem Temporal

Esquerda: A Analogia



Se todos os estágios levarem o mesmo tempo, uma nova carga é finalizada a cada ciclo, mesmo que o processo total leve vários ciclos.

Direita: Mapeamento Técnico

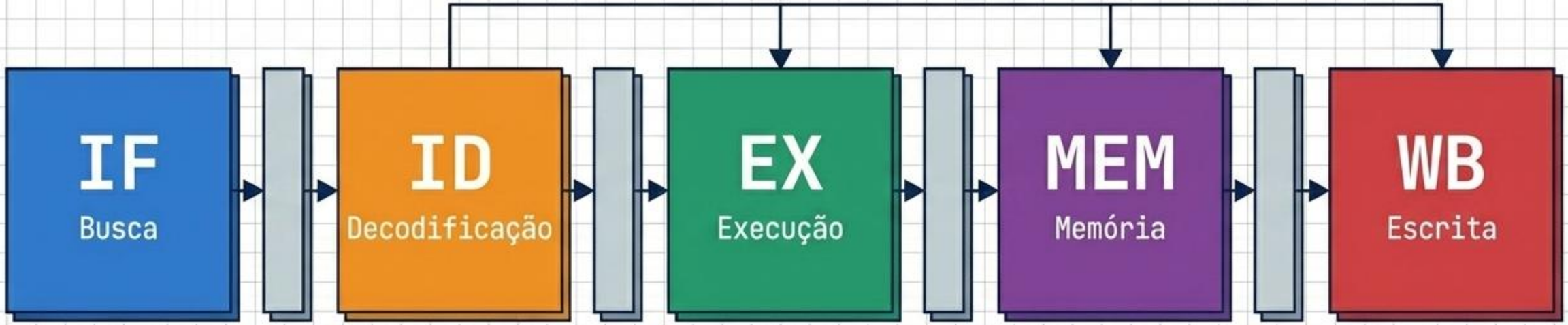


O pipeline não pula etapas; ele sobrepõe a execução de diferentes instruções em estágios distintos.

Insight: O hardware é dividido em unidades independentes. A chave do paralelismo é manter todas as unidades trabalhando 100% do tempo em instruções sequenciais diferentes.

Gemini Notebook

O Pipeline Clássico de Cinco Estágios



Busca a instrução na memória e incrementa o PC.

Decodifica a instrução e lê os registradores independentemente.

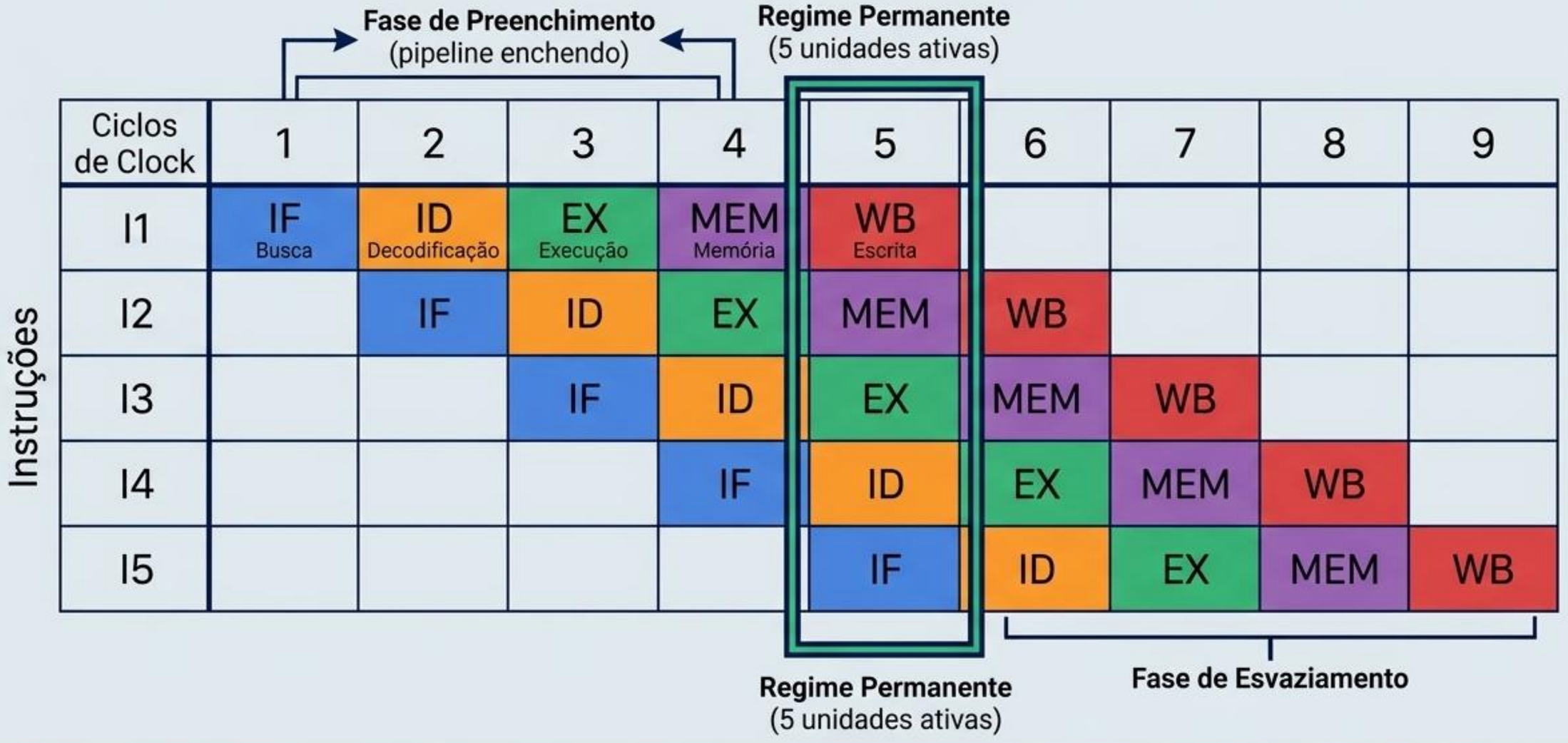
Executa a operação na ULA ou calcula endereços.

Acessa a memória de dados (leitura ou escrita).

Escreve o resultado final de volta no registrador destino.

Gemini Notebook

O Diagrama Temporal: Execução Ideal



A partir do ciclo 5, concluímos 1 instrução a cada ciclo, independentemente do tempo total da instrução.

Gemini Notebook

Pipeline torna cada instrução individual mais rápida? **Não.**

Latência (O Tempo)



O tempo de execução de UMA instrução permanece o mesmo (ou até aumenta levemente devido aos registradores).
Ex: 5 estágios $\times$ 2ns $=$ 10ns de latência.

Throughput / Vazão (A Quantidade)



O que melhora exponencialmente é a vazão de saída.

Ex: Com clock de 2ns, o pipeline entrega 1 instrução a cada 2ns $=$ 500 milhões de instruções por segundo.

Conclusão:
O pipeline otimiza o volume total de trabalho entregue pelo processador, não a velocidade do item individual.

Gemini Notebook

A Realidade Imperfeita: Introdução aos Hazards

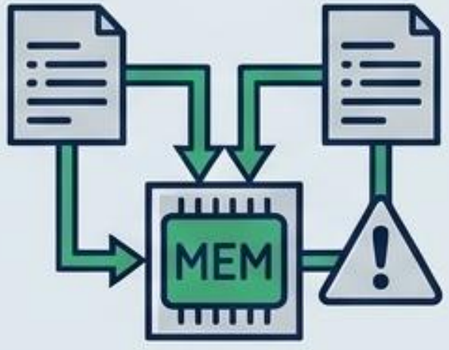
O fluxo contínuo ideal é interrompido por **Hazards** — obstáculos que impedem o avanço das instruções. O hardware reage inserindo **Bolhas** (Stalls) no pipeline.

Stall (Bolha) - Desperdício de Ciclo

Hazards Estruturais

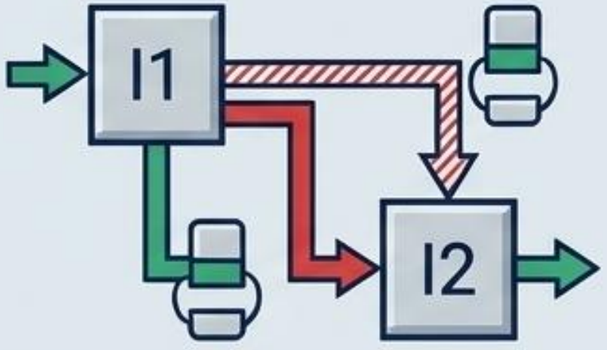
O hardware **não suporta a combinação de instruções.**

Ex: Disputa por uma única memória para ler dados e buscar instruções ao mesmo tempo.



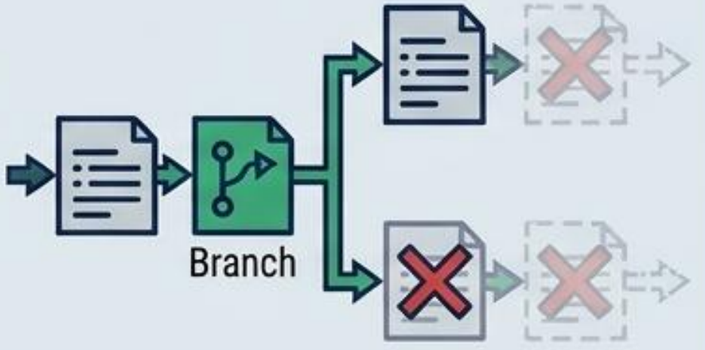
Hazards de Dados

Dependências lógicas.
Uma instrução precisa ler um un dado que a instrução anterior ainda não terminou de calcular.



Hazards de Controle

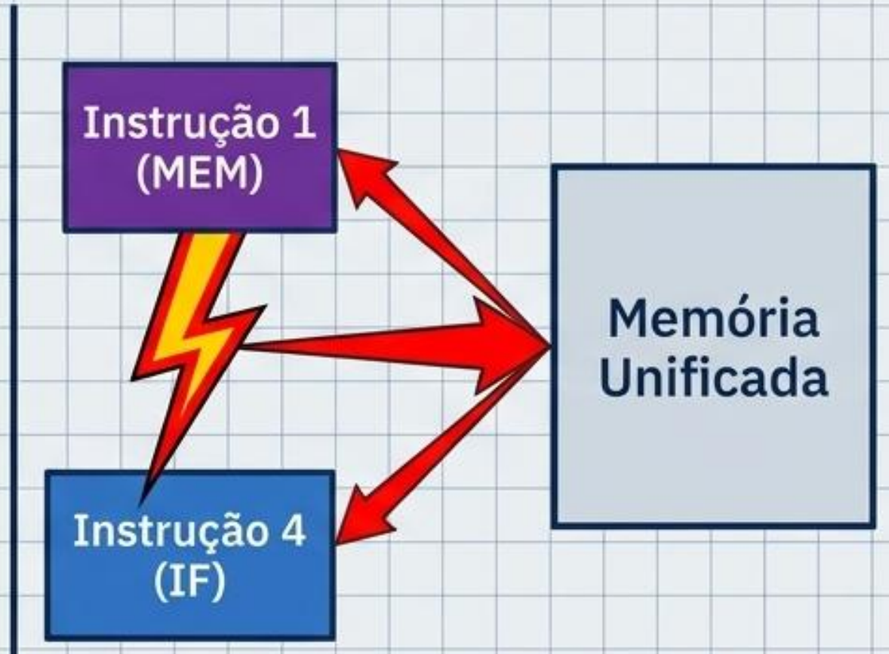
Quebras de fluxo.
Uma instrução de desvio (Branch) altera o fluxo, invalidando as instruções que já entraram no no pipeline equivocadamente.



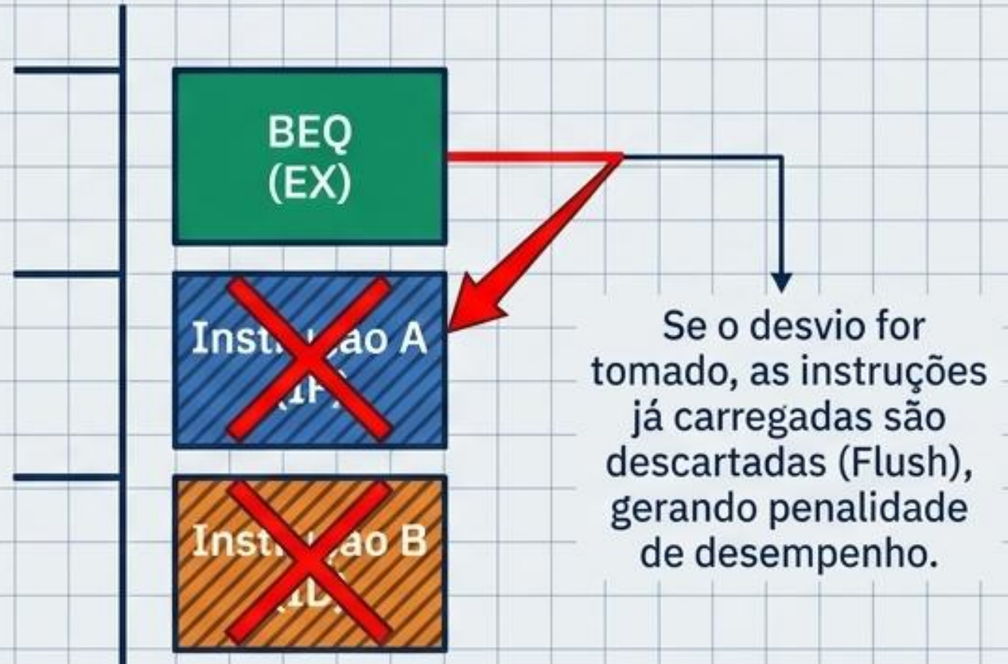
Gemini Notebook

Exemplos Práticos de Pipeline Hazards

Conflito de Recursos (Estrutural)



Descarte por Desvio (Control Hazard / Flush)

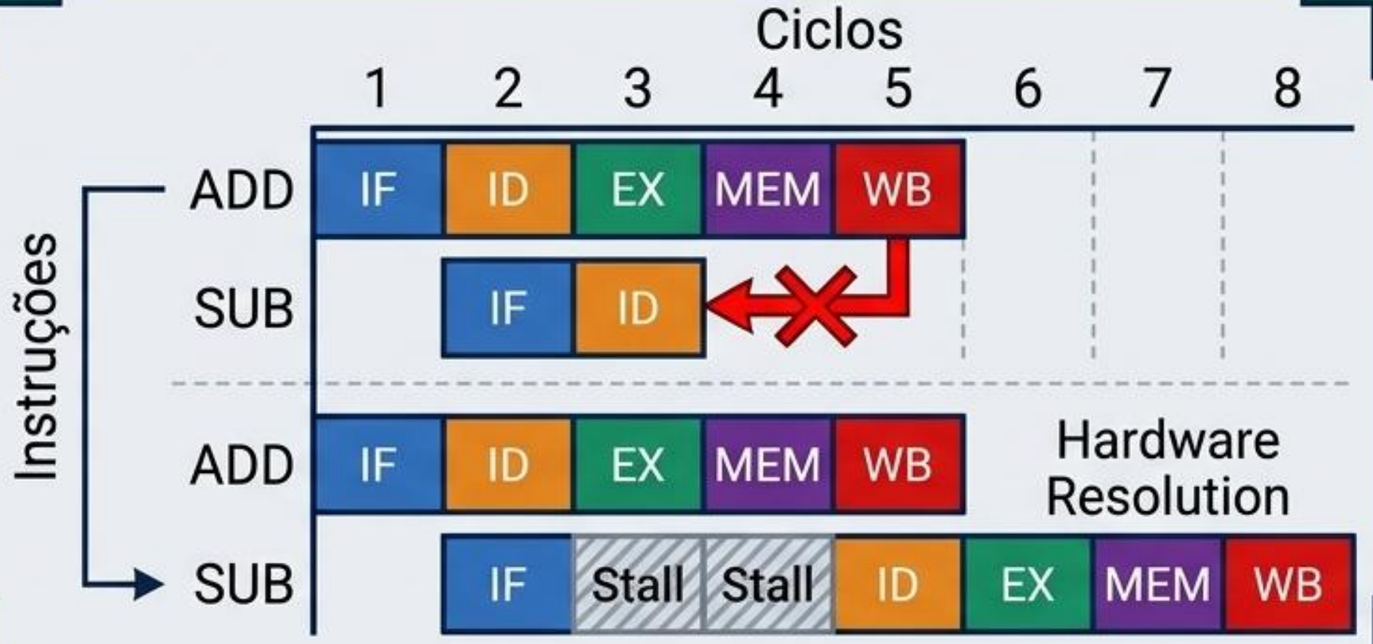


Gemini Notebook

Hazards de Dados (RAW) e a Inserção de Bolhas

```
ADD $s0, $t0, $t1
SUB $t2, $s0, $t3
```

O SUB precisa ler $s0, mas o ADD só escreve o resultado no final.



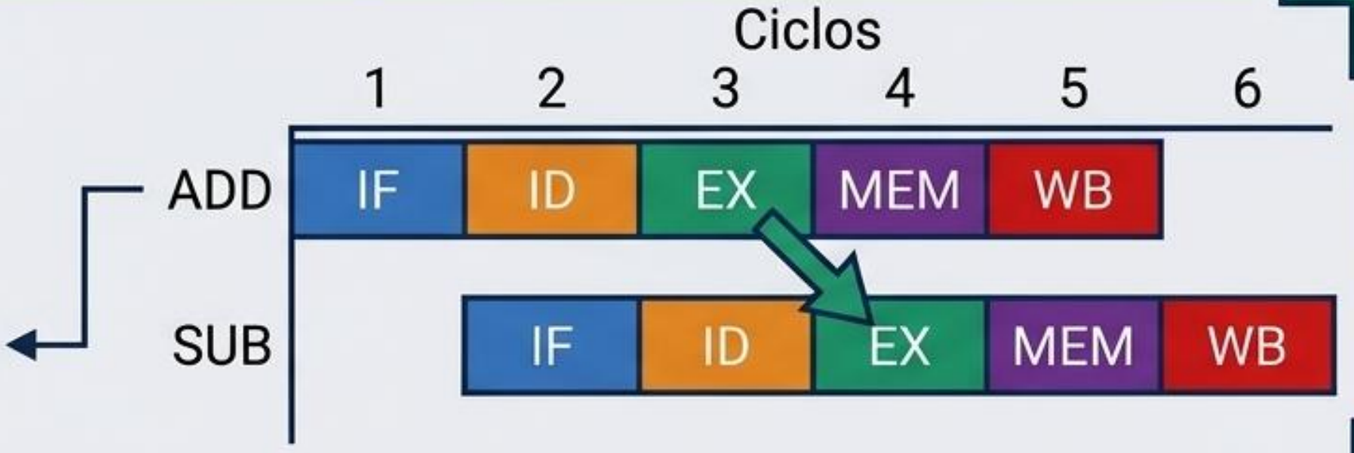
Dependência de dados é uma característica do programa. Um Hazard efetivo só ocorre quando o hardware falha em mascarar essa dependência, resultando em **Stalls (bolhas cinzas)**.

Gemini Notebook

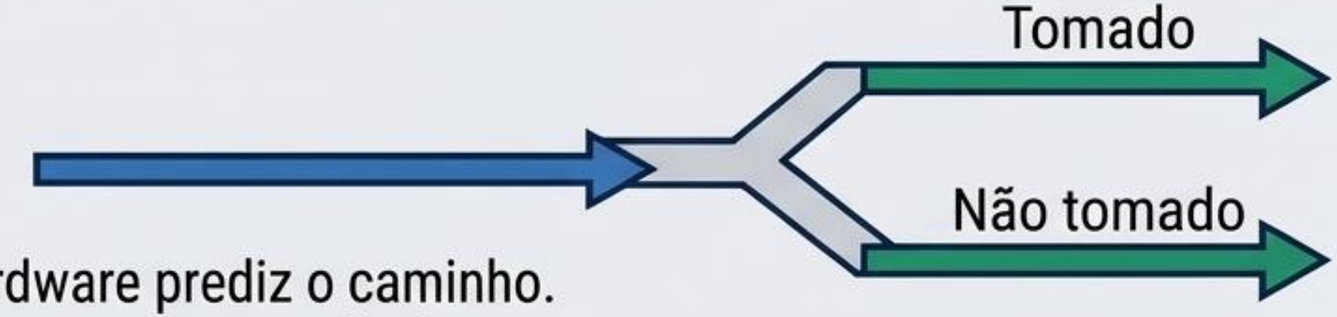
Técnicas de Mitigação: Salvando o Desempenho

Bypassing / Forwarding (Avanço de Dados)

O hardware intercepta o dado na ULA e o envia diretamente para a próxima instrução, eliminando bolhas.



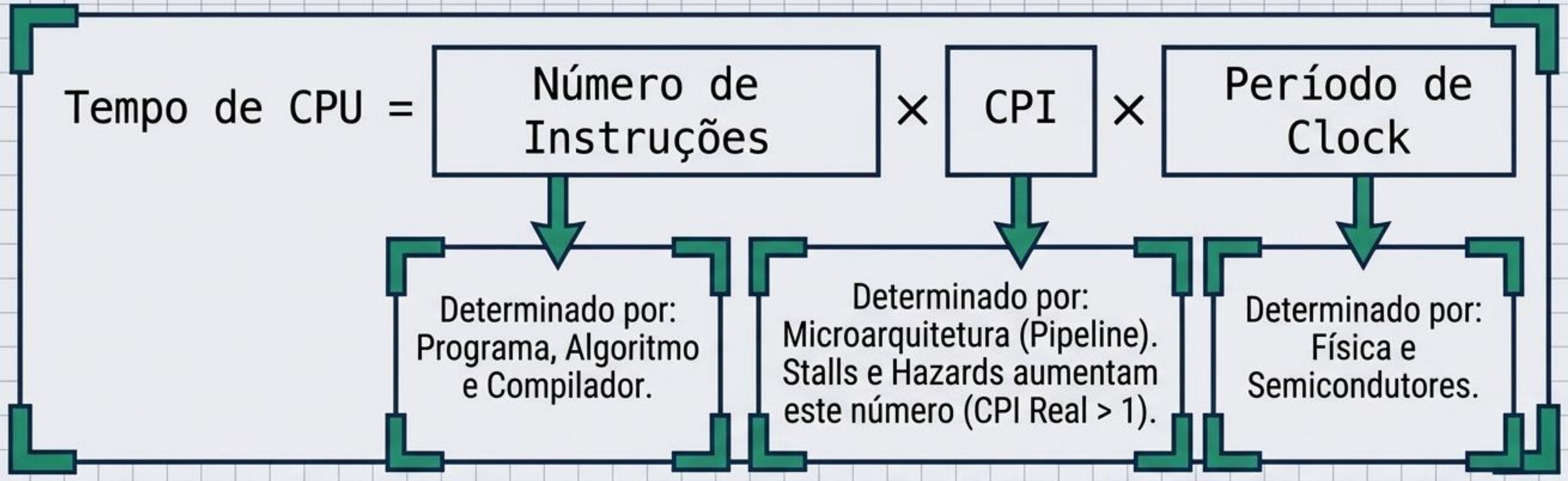
Predição de Desvios (Branch Prediction)



Em vez de pausar o pipeline, o hardware prediz o caminho. Acertos mantêm o fluxo contínuo; erros causam flush.

Gemini Notebook

A Verdade Matemática: A Equação do Desempenho



O Mito do Megahertz

Aumentar a frequência do clock piorando a arquitetura (inflando o CPI com bolhas) resulta em **zero ganho real** de desempenho.

Gemini Notebook

Cálculo Prático: O Impacto dos Stalls no CPI

O Cenário

Programa de 1.000 instruções. Pipeline ideal (CPI = 1).
Contudo, 20% das instruções sofrem penalidade de desvio de 2 ciclos.

```
Step 1: Formula -> CPI_Real = CPI_Ideal + (Frequência de Penalidade × Custo do Stall)
Step 2: Substitution -> CPI_Real = 1.0 + (0.20 × 2)
Step 3: Result -> CPI_Real = 1.4
```



Insight: Os hazards tornaram a execução total 40% mais lenta do que a teoria prometia.

Gemini Notebook

Avaliação de Melhoria: Cálculo de Speedup

Máquina A
(Sequencial)

Ciclo: 10ns

CPI: 1

Tempo:
10 ns/instrução

$$\text{Speedup} = \frac{\text{Tempo A}}{\text{Tempo B}} = \frac{10}{3.0} = 3.33x$$

Máquina B
(Pipeline de 5 Estágios)

Ciclo: 2.5ns
(overhead de
registradores)

CPI Real: 1.2
(devido a stalls)

Tempo:
3.0 ns/instrução

Crucial Insight:

Apesar de ter 5 estágios, o Speedup real é apenas 3.33x (não 5x). O aumento da latência do ciclo e a inserção de bolhas impedem o ganho teórico perfeito.

Gemini Notebook

Mapa Conceitual da Arquitetura

Microarquitetura: Pipeline e Mitigações

Reduz o CPI e minimiza bolhas estruturais/controle.

Tecnologia e Física: Frequência de Clock

Determina o tempo base do ciclo; depende de estágios equilibrados.

Desempenho Global (Tempo de Execução)

Software: Compilador

Reordena instruções invisivelmente para evitar dependências e Hazards.

Sistema de Memória: Hierarquia/Cache

Minimiza stalls massivos.

Sem cache, o pipeline esperaria ciclos infinitos pela RAM.

Gemini Notebook

Síntese Final: O Que Devemos Levar Desta Unidade?

UNEMAT 1

Latência vs. Vazão

O pipeline aumenta drasticamente a Vazão (Throughput) de instruções, mas não reduz o tempo de processamento individual de nenhuma delas.

UNEMAT 2

O Perigo das Bolhas

Dependências lógicas sempre existirão no código. O desafio é usar arquitetura para evitar que virem Hazards Efetivos.

UNEMAT 3

O Mito do Clock

Frequências altíssimas não garantem desempenho se o pipeline estiver paralisado (alto CPI). Desempenho exige equilíbrio.

UNEMAT 4

A Orquestração Global

O tempo ótimo da CPU depende da perfeita sinergia entre Compilador, Microarquitetura e Eletrônica.

Gemini Notebook